TEMA 1 INTRODUCCIÓN A LA INGENIERÍA DEL SOFTWARE OO

Tema 1
Introducción a la ingeniería del software orientado a objetos

1. Qué es la ingeniería del software
2. El ciclo de vida del software
3. Desarrollo estructurado y desarrollo orientado a objetos
4. Las herramientas CASE
5. El OMG y el UML

1. QUÉ ES LA INGENIERÍA DEL SOFTWARE

Un **software** no es una obra de arte, sino un producto de consumo utilitario y masivo; sin embargo es un producto industrial con algunas características especiales: es un producto singular (aunque hay copias de software usados por millones de personas, la producción en serie no nos interesa y no suele ser algo excesivamente generalizado) y no se estropea con el paso del tiempo.

La **ingeniería del software** comprende las técnicas, métodos y herramientas que se utilizan para producirlo. Hoy día la calidad del software y su productividad no han alcanzado un punto óptimo. Por ejemplo, no podemos probar un producto software frente a cualquier posible condición de utilización, lo cual va en contra de su calidad; y en cuanto a la productividad recalcar que cuando diseñamos un software, solemos partir de cero, no como cuando se diseña un coche, donde ya se tienen piezas perfectamente testadas y homologadas (tornillos, chapa, etc) que casi con seguridad sabemos no darán ningún tipo de error en el producto final. Actualmente ya se está intentando conseguir un cierto grado de reutilización de software con el diseño orientado a objetos.

Ciclo de vida clásico

2. EL CICLO DE VIDA DEL SOFTWARE

El ciclo de vida del software lo constituyen las etapas que preceden, y las que siguen a la etapa de programación. Existen varios ciclos de vida distintos, con etapas bien definidas, esto es así porque pretendemos cumplir los plazos dados, respetar los límites de los recursos asignados y también ofrecer calidad.

2.1. El ciclo de vida clásico

También denominado ciclo de vida en cascada, tiene las siguientes etapas:

- **Análisis previo**: Se definen aquí los grandes rasgos del sistema de software que tendrá que dar soporte informático a unas determinadas actividades. Deberá funcionar en un entorno hardware y red determinado que habrá que indicar; contaremos también con los recursos necesarios para el desarrollo y los condicionamientos temporales. El documento resultante de este análisis es la especificación del sistema y sirve de base para la siguiente etapa.
- **Análisis de requisitos**: Definiremos aquí con detalle las necesidades de información que tendrá que resolver el software, sin tener en cuenta por el momento, los medios técnicos. Deberemos requerir de los futuros usuarios las funciones y requisitos en general; y con esta información redactaremos la especificación de requisitos, con la suficiente precisión para que se pueda desarrollar y servir de base para un contrato entre el equipo de desarrollo del software y sus futuros usuarios.
- **Diseño**: El diseño especifica la solución a cada uno de los requisitos de la fase anterior. Con ello recogemos el documento especificación de diseño, que será la base para la programación.
- **Programación**: Traducimos a código el diseño de la fase anterior.

- **Prueba**: Testeamos el software desde distintos puntos de vista tratando de localizar y corregir en el software y en la documentación todos los posibles errores.
- **Mantenimiento**: o explotación del software, constantes actualización para mejorar la eficiencia o añadir funciones. Este proceso puede durar hasta 10 años o más, por lo que el coste de esta fase suele ser de 2 a 5 veces mayor que el coste de desarrollo.

No obstante, este ciclo ha sido muy criticado porque tiene diferentes inconvenientes; el principal es que los requisitos iniciales que recogemos en el análisis de requisitos muchas veces no son fiables o son incompletos. Es muy difícil encontrar un conjunto de futuros usuarios que conozcan el entorno del software y que sepan exactamente qué es lo que quieres que haga el software. Así que una vez terminada oficialmente la fase de análisis y comenzada la de diseño, seguirán surgiendo nuevos requisitos y cambios; con lo que el software y la documentación pierden validez. Por ello, se han definido otros tipos diferentes de ciclo de vida.

2.2. El ciclo de vida con prototipos

Consiste en un software provisional que prioriza la facilidad y rapidez en la modificación sobre la funcionalidad; solo sirve para que los usuarios puedan ver cómo sería el contenido y el posible funcionamiento y apariencia del software, sin realizar realmente estas funciones. Así por comparación, los futuros usuarios podrán definir más exactamente qué es lo que quieren; y por supuesto se puede volver a fabricar otro prototipo y otro más hasta afinar exactamente en el resultado que buscamos.

2.3. La programación exploratoria

Se elabora una primera versión del software o de una parte de éste, se le enseña a los futuros usuarios para que la critiquen y se hacen los cambios que estos sugieran, esto se repetirá tantas veces como sea necesario. La diferencia con los prototipos es que en la programación exploratoria, el programa sí funciona y no es un mero prototipo (viene a ser lo que realiza Microsoft con sus betatesters, usuarios a los que les ofrece el programa para que busquen errores y modificaciones, antes de sacarlo a la venta comercialmente en una versión definitiva).

2.4. El ciclo de vida del Rational Unified Process

Distingue 4 etapas:

- Inicio: Se establece la justificación económica y el alcance del proyecto.
- Elaboración: Se estudia el alcance del software, a quién debe dar cobertura y se realiza una planificación del proyecto.
- Construcción: Se desarrolla todo el proyecto de forma iterativa e incremental.
- Transición: Se entrega el producto al cliente y comprende todo el comienzo de su utilización , realizando los cambios necesarios y oportunos.

3. DESARROLLO ESTRUCTURADO Y DESARROLLO ORIENTADO A OBJETOS

Los métodos estructurados provienen de la programación estructurada (C, C++), mientras que los métodos orientados a objetos provienen de la programación orientada a objetos (Java). Hay dos características del desarrollo orientado a objetos que han favorecido su expansión: por un lado permite por vez primera la **reutilización de software** de manera significativa, con lo que mejoramos la productividad y calidad a la que hacíamos referencia en el primer punto de este tema; y por otro lado se facilita **la ayuda al desarrollo** sobre todo por la notación orientada a objetos UML mayoritariamente aceptada.

4. LAS HERRAMIENTAS CASE

CASE significa Computer-Aided Software Engineering. Son, por tanto, software de apoyo al desarrollo, mantenimiento y documentación informatizados del software. Se excluyen como herramientas CASE aquellas que no tienen solo finalidad (procesadores de texto, hojas de cálculo… que si bien necesitamos, no podemos considerar CASE) o las que se utilizan para codificar software. Así incluimos en las herramientas CASE las herramientas diagramáticas (a través del dibujo reconocen una clase, y no un simple rectángulo gráfico), las herramientas de gestión de la prueba y de la calidad y la de gestión de cambios.

La expansión de las herramientas CASE se vio bastante frenada en su día debido a la disparidad y diversidad (no estandarización) de las técnicas que se utilizaban; con el diseño orientado a objetos hemos vuelto a la estandarización estas herramientas se expanden rápidamente.

5. EL OMG Y EL UML

El OMG (Object Management Group) se creó en 1989 para fomentar el uso de la tecnología orientada a objetos e impulsar la generación de este tipo de software; es una organización no lucrativa y está constituida por más de 800 empresas.

El UML (Unified Modeling Language) es un modelo para la construcción de software orientado a objetos, que ha sido propuesto como estándar de ISO por el OMG. Con él se ha llegado aun modelo orientado a objetos único de forma oficial, pero eso no quiere decir que todo el proceso en todos los ingenieros sean únicos. De momento se ha conseguido que haya unos diagramas que todos los desarrolladores entenderán y harán de la misma manera; lo cual supone un importante avance, pero no es definitivo.

TEMA 2 UML1: El modelo estático

Tema 2
UML1: El modelo estático

1. Concepto de modelo estático y diagrama de clases
2. Clasificadores y paquetes
3. Clase y conceptos afines
4. Relaciones entre clases
5. Comentarios y restricciones

1. CONCEPTO DE MODELO ESTÁTICO Y DIAGRAMA DE CLASES

El modelo estático del UML es aquél en el que se describen las clases y los objetos. Se denomina estático porque muestra todas las relaciones posibles a lo largo del tiempo, no las que son válidas en un cierto momento. El modelo estático consta de los dos diagramas siguientes:

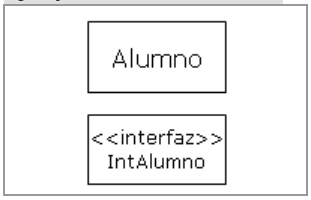
- **Diagrama de clases**: Que puede contener clases y objetos y relaciones entre estos, y que se hace siempre. Este diagrama muestra la estructura estática de clases en un dominio.
- **Diagrama de objetos**: Que solo contiene objetos y relaciones entre éstos, y es opcional.

El modelo estático pretende ser independiente del lenguaje de programación, por lo que será responsabilidad del diseñador del software evitar caer en la utilización de conceptos no soportados por el lenguaje de programación.

2. CLASIFICADORES Y PAQUETES

El **clasificador** es la entidad básica del modelo estático. Un clasificador es más general que una clase. El clasificador en sí mismo no tiene símbolo gráfico, sino que lo tienen sus estereotipos; clases, tipos de datos, interfaz. La utilidad de este clasificador radica en el hecho de que los estereotipos mencionados tienen mucho en común, por lo que es suficiente con realizar la indicación una vez en el clasificador. La notación gráfica es la misma para todos: un rectángulo. Todos los clasificadores deben tener un nombre. En un clasificador se puede indicar la palabra clave "estereotipo" entre comillas latinas; cuando no se indique ningún estereotipo, se tratará de una clase.

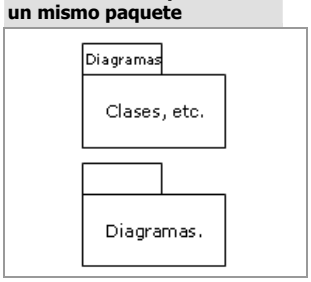
Ejemplo de clase e interfaz



Un **paquete** es solo una caja que contiene elementos, que pueden ser clasificadores, objetos, u otros paquetes… Se representa como se observa en la figura lateral. Las relaciones que se pueden dar entre paquetes son:

- **De especialización**: si un paquete hereda de otro, el primero tiene elementos más restrictivos del segundo.
- **De inclusión**: Todos los elementos de un paquete están contenido en otro (éste además puede tener muchos más paquetes).
- **De importación**: Desde el paquete que importa, se reconocen los nombres de los elementos del otro paquete visibles desde el exterior.
- **De acceso**: No solo se reconocen los nombres como antes, sino que además se pueden utilizar.

Dos formas de representar un mismo paquete



3. CLASE Y CONCEPTOS AFINES

Una **clase** describe un conjunto de objetos en el que todos tienen los mismos atributos y operaciones. Cuando esa "clase" se palpa, se instancia, tenemos un **objeto** individual.

La **representación de una clase** es más compleja que la de un clasificador, dado que consiste en un encapsulado de atributos y operaciones, daremos una representación gráfica más detallada que un simple rectángulo. Consistirá pues en un rectángulo con 3 compartimentos. En el primero estará el nombre de la clase, en el segundo la lista de atributos y en el tercero los servicios de la clase.

- **Compartimento del nombre**: Se puede indicar en la parte superior un estereotipo y justo debajo el nombre de la clase; se recomienda que se use un sustantivo singular que a veces puede tener una segunda palabra que la

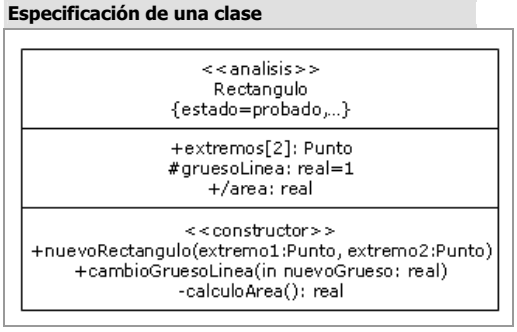
califique y también se recomienda que empiece por mayúscula, podemos encontrar debajo del nombre comentarios optativos entre llaves denominados cadenas de caracteres de propiedades o valores etiquetados, si hay puntos suspensivos detrás implican que hay más elementos que no se ven. En el ejemplo lateral, el estereotipo análisis tiene una clase rectángulo, que se ha identificado en la etapa de análisis y tiene más etiquetas que no alcanzamos a ver.

- **Compartimento de los atributos**: Cada atributo de la clase tiene varios aspectos a tratar, por orden en su especificación son:

  Visibilidad: Indica hasta qué punto las operaciones de otras clases pueden acceder al atributo, los símbolos que podemos encontrar son: público (+), protegido (#), privado (-) y dentro del paquete (~).

  Nombre: Se recomienda que empiece por mayúscula, si es derivado (es decir, redundante con otros a partir de los cuales se puede obtener el valor), el nombre tiene que ir precedido de /.

Especificación de una clase



Expresión del tipo: real, entero, string…

Valor inicial si lo tuviera

Property string igual que en el nombre de la clase.

- **Compartimiento de las operaciones**: Tiene a su vez también varios aspectos a tratar:

Visibilidad

Nombre: Se recomienda que empiece por minúscula.

Lista de parámetros: se compone de parámetros separados por comas, y a su vez, cada parámetro se define con el tipo (in, out o inout; o el tipo de retorno cuando devuelva un valor como resultado), la expresión del tipo y el valor por omisión

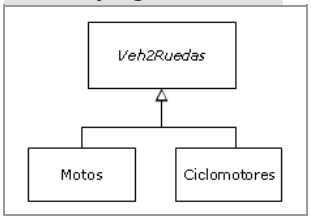
La herencia en el análisis y el diseño

La herencia presupone que existen dos clases, de las cuales una es la superclase y la otra la subclase. Esta subclase comprende un conjunto de los objetos de la superclase con todos los atributos y operaciones de instancia de la superclase y además, puede tener algunos específicos de la subclase. La herencia puede suceder por:

Herencia por especialización

- **Herencia por especialización**: Se crea una subclase más especializada y restrictiva a partir de la superclase definida con anterioridad. Es ejemplo de ello la subclase "suite" de la superclase "habitación"; una suite es obviamente una habitación, pero tiene características específicas de espacio, decoración, tamaño. En cambio no todas las habitaciones son suites. La suite (subclase) es más restrictiva y menos generalizada que la habitación (superclase).

Herencia por generalización



- **Herencia por generalización**: A partir de varias subclases, se encuentra una superclase que, solo se puede instanciar por medio de las subclases que la constituyen. Por ejemplo, en un municipio a la hora de recaudar se dan cuenta que las motos y ciclomotores cumplen prácticamente las mismas características salvo alguna diferencia. A partir de las dos se crea la superclase Vehiculosde2ruedas, que se subdivide en motos o ciclomotores. Todos los vehículos de dos ruedas o son motos o ciclomotores, nunca ambos (disjoint) y así vehiculosde2ruedas se convierte en una **clase abstracta** de la cual no se pueden crear (instanciar) directamente objetos, sino que se tienen que crear necesariamente en alguna de sus subclases. Se indica una clase abstracta bien poniendo su nombre en cursiva o bien con la propiedad {abstract} en el compartimento del nombre. Una clase abstracta puede tener operaciones abstractas que son las que solo están implementada en las subclases, en principio, en forma diferente en cada una.

Variantes en el concepto de clase

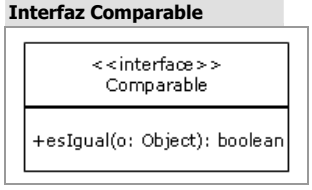
Encontramos clases **diferidas** que con clases abstractas que tienen alguna operación abstracta; clases **terminales**, de las que no pueden colgar ninguna subclase, no nos suele interesar que herede ninguna otra clase de ellas por varios motivos; **metaclases**, son clases cuyas instancias son a su vez, también clases y no objetos; **clases parametrizadas** o también llamadas plantillas es un descriptor

de clase frmalmente igual a una clase excepto que algún término de su definición es un parámetro. Cuando se dan valores a todos los parámetros de una plantilla, se obtiene una clase totalmente especificada; **clases de utilidad** son aquellas que incluyen rutinas y datos que no corresponden a ninguna clase u operación (por ejemplo parámetros del sistema que sólo pueden tener un valor cada uno), estas clases de utilidad no tendrán objetos.

Interfaces

Una interfaz describe un conjunto de operaciones visibles de una clase, sin indicar su implementación. Se dice que dicha clase en cuestión implementa la interfaz; una interfaz no es una clase, pero equivale a una clase abstracta sin atributos y con todas sus operaciones diferidas. Puede las interfaces estableces relaciones de herencia entre sí, pero no pueden participar en asociaciones ni tener estados. La notación de la interfaz es como la de la clase, pero con el estereotipo interfaz y sin el compartimento de atributos, porque no los tiene.

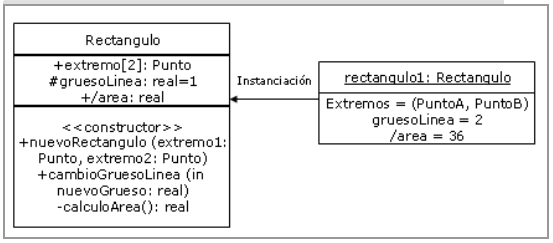
Interfaz Comparable



Representación de los objetos

Un objeto se representa gráficamente de una manera muy parecida a las clases: se indican los valores en los atributos de instancia y, opcionalmente, un nombre en el objeto, el nombre de la clase, todo subrayado. Se suelen omitir el tipo de los atributos, así como el compartimento de las operaciones, dado que ya se conocen gracias a la especificación de la clase.

Instanciación



4. RELACIONES ENTRE CLASES

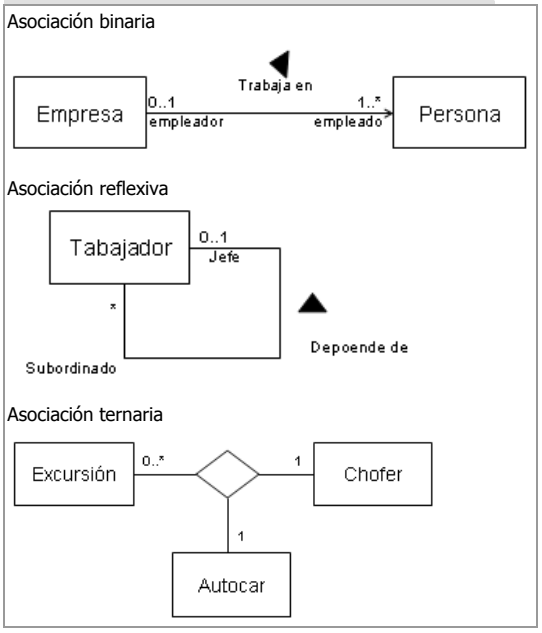
Las relaciones entre clases que vamos a estudiar son las siguientes:

- Asociaciones: binarias, reflexivas, n-arias.
- Clases asociativas: asociaciones calificadas, alternativas y derivadas.
- Agregaciones y composiciones: agregaciones, composiciones.
- Relaciones de dependencia

Asociaciones

Hay una asociación entre clases cuando una clase necesita otra u otras para la implementación de sus operaciones, lo cual se cumple por medio del paso de mensaje entre estas. Cada papel que se establece en esta asociación tiene asociada una cardinalidad. Estudiamos los diferentes tipos:

Asociaciones



- **Asociaciones binarias**: Son las que tienen lugar entre dos clases; como podemos observar en el ejemplo lateral observamos que una empresa se relaciona con las personas cuando son sus empleados. La flecha abierta implica que podemos navegar por los empleados a partir de la empresa. La cardinalidad 0..1 nos indica que una persona puede estar en 0 o en 1 empresa (nada de tener dos trabajos) y 1..* implica que toda empresa, al menos tiene un empleado.
- **Asociaciones reflexivas**: Es un tipo especial de relación binaria en la que la clase se relaciona consigo misma. En el ejemplo lateral vemos que un trabajador a la vez puede ser jefe y subordinado. En este caso puedo tener cualquier número de subordinados, incluso 0 (*) y cada trabajador puede tener un jefe o ninguno (0..1).
- **Asociaciones n-arias**: Son aquellas en las que se relacionan n números de clases. Para facilitarlo, estudiaremos las ternarias. En el ejemplo lateral observamos como un autocar y un chofer pueden realizar múltiples excursiones

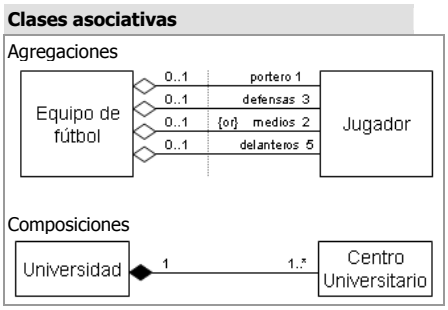
o ninguna; pero en una excursión en concreto contamos con un único autocar y un único chofer.

Clases asociativas

En principio, una asociación no es una clase; pero si ésta tiene que tener atributos y operaciones propias, entonces debe definirse como clase; y encontramos los siguientes tipos:

- **Asociaciones calificadas**: Se establece una asociación calificada mediante un atributo de una clase. Veámoslo con un ejemplo. En el ejemplo lateral observamos que en cada departamento y para cada categoría laboral, puede haber un número cualquiera de empleados (incluído 0). Esto lo podemos categorizar como el ejemplo inferior. Del atributo categoría de la clase departamento, pende la relación con el número de empleados.
- **Asociaciones alternativas**: Cuando una clase participa en dos asociaciones o más, pero cada objeto concreto solo puede pertenecer a una de ellas, hablamos de asociaciones alternativas (disjoint) y la forma de representarlo lo tenemos en el ejemplo lateral.
- **Asociaciones derivadas**: Es una asociación redundante que se puede obtener como combinación de otras relaciones del modelo. En el ejemplo, los alumnos matriculados de un curso concreto, los obtenemos por redundancia y por ello se pondría / que es el indicador de elemento derivado.

Agregaciones y Composiciones

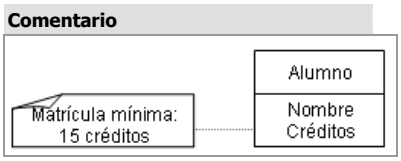


- **Agregaciones**: Es un caso particular de asociación binaria en la que uno de los papeles tiene el significado de "parte" y otro el de "todo". Pueden tener diferentes significados: acoplamiento-piezas (una máquina y sus piezas, un circuito eléctrico y sus componentes); continente-contenido (avión y los pasajeros, el avión seguirá siéndolo aunque no contenga pasajeros); colectivo-miembros (como el caso del equipo de fútbol que vemos en el ejemplo lateral). Un objeto puede tener diferentes componentes y éstos se señalan con un rombo hueco. En el ejemplo, observamos como un equipo de fútbol está formado por 1 portero, 3 defensas, 2 medios y 5 delanteros; se entiende que cada delantero es intercambiable. A su vez, un jugador puede estar en 1 equipo de fútbol o en ninguno (se acabó el fatigarse jugando en varios equipos de forma simultánea).
- **Composiciones**: En una composición, los objetos componentes no tienen vida propia, sino que cuando se destruye el objeto compuesto del que forman parte, también se destruyen ellos. Un objeto componente solo puede formar parte de un objeto compuesto y no puede pasar de un objeto compuesto a otro. Por ejemplo, un centro universitario pertenece a una universidad (y solo a una); en caso de que esta Universidad cierre, el centro universitario también lo hará, no podrá pasar a depender de otra. Las composiciones se representan con un rombo opaco.

Relaciones de dependencia

En esta relación, un elemento del modelo (cliente) depende para su implementación o funcionamiento de otro elemento (suministrador). El símbolo de una relación de dependencia es una flecha de línea discontinua y punta abierta.

5. COMENTARIOS Y RESTRICCIONES



Los **comentarios** en los modelos UML se ponen dentro de un rectángulo, con un vértice doblado enlazado con una línea discontinua al elemento al que se refiere ese comentario.

Las **restricciones** expresan condiciones que debe cumplir el elemento del modelo al que se asocian, se representan igual que los comentarios, pero entre llaves {}, de modo que puedan ser interpretadas por las herramientas

CASE. Como ya sabemos de otras asignaturas, en la programación por contrato hay 3 tipos de restricciones:

- **Precondiciones**: Son restricciones que deben cumplirse antes de la ejecución de una operación.
- **Postcondiciones**: Deben cumplirse al acabar la ejecución de una operación.
- **Invariantes**: Deben cumplirse en todo momento.

TEMA 3 UML2: EL MODELO DINÁMICO Y DE IMPLEMENTACIÓN

Tema 3
UML2: El modelo dinámico y de implementación

1. El diagrama de estados
2. El diagrama de casos de uso
3. Los diagramas de interacción
4. El diagrama de actividades
5. Los diagramas de implementación

1. EL DIAGRAMA DE ESTADOS

Hay objetos cuyo comportamiento puede variar a lo largo del tiempo; cada una de estas variaciones son estados y son muy importantes en determinadas aplicaciones, como pueden ser las de tiempo real. En el diagrama de estados distinguimos los siguientes conceptos:

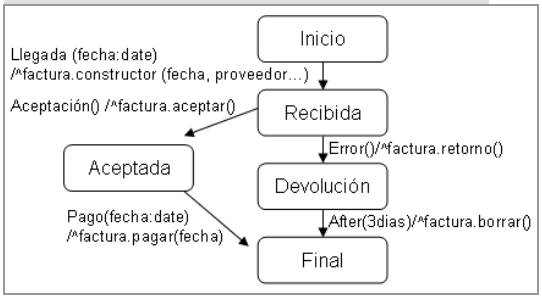
- **Estado**: Es una situación determinada dentro de la vida de un objeto; eso sí, un estado no corresponde a un instante de tiempo, sino que dependerá de que se cumpla alguna condición o se produzca un acontecimiento.
- **Transición simple**: Consiste en que el objeto para de un estado (origen) a otro (destino), que podría incluso ser el mismo. Esta transición comienza cuando se produce un acontecimiento determinado y, a la vez, se cumple una condición específica (**condición de guarda**).
- **Transición interna**: Son pseudotransiciones en las que no hay cambios de estado; se usan para especificar acciones que se deben ejecutar en respuesta a acontecimientos que no provocan cambios de estado.
- **Acción**: Es un proceso atómico, es decir, que o no se ejecuta o se ejecuta hasta el final.
- **Acontecimiento**: Son hechos que pueden provocar transiciones de un estado a otro. Encontramos los acontecimientos de **llamada** (cuando se llama una operación del objeto al que corresponde el diagrama), de **señal** (recepción de una señal por un objeto), de **cambio** (notificación de que una condición ha llegado a ser cierta) y de **tiempo** (o ha pasado un período de tiempo o que es una hora determinada).
- **Acontecimiento interno**: Son pseudoacontecimientos que están ligados a un estado en lugar de a una transición. Hay tres tipos: de **entrada** (Se producen cuando el objeto entra en el estado correspondiente y tienen como nombre la palabra clave entry), de **salida** (Se producen a la salida del estado, su palabra clave es exit) y **de acción** (especifican acciones que se ejecutan cuando se llega al estado en cuestión y acaban por sí mismas o cuando se sale del estado)

Las transiciones se representan mediante flechas de punta coloreada que van del estado de salida al de llegada. Con la flecha se encuentra una expresión que presenta la siguiente sintaxis:

Signatura [guarda] / acción ^ envío

La **signatura** normalmente tiene un formato que reúne el nombre del acontecimiento y entre paréntesis los parámetros con su nombre y su expresión. Tipos especiales de acontecimientos son aquellos que son de tiempo –after()- o –when ()-. La **guarda** es una expresión que puede tomar el valor verdadero o falso. La **acción** se especifica en pseudocódigo y deberá incluir la clave defer como primera acción si el acontecimiento es diferido. El **envío** presenta la siguiente forma: destino.mensaje (argumentos).

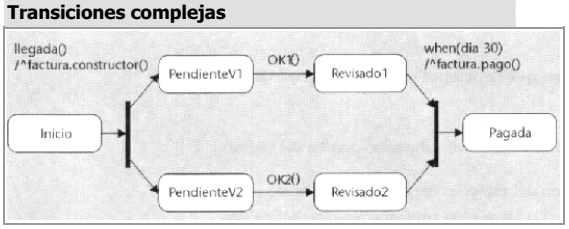
Diagrama de estado



Así, en el diagrama lateral, observamos que una factura que llega, si es aceptada, pasará al estado aceptada y de él, se procederá a su pago. En cambio, si da algún tipo de error (no especificado), pasará a un estado devolución, en el que permanecerá hasta que pasen 3 días y entonces se procederá a borrar la factura en cuestión.

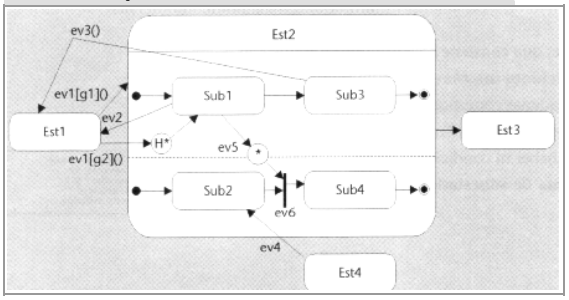
Pueden existir **transiciones complejas** dado que un objeto o interacción puede estar en más de un estado al mismo tiempo, y pueden haber transiciones que salgan de más de uno o que vayan a parar a más de uno. Suele realizar funciones de sincronización. Se representan con barras cortas y gruesas de las que salen o a las que llegan los diferentes estados. Así, en el ejemplo lateral vemos que cuando llega una factura, pasamos a poner en pendiente lo que llega y lo que se pidió. Se comparan estos dos ítems (V1 y V2) y si se aceptan, la factura será pagada.

Transiciones complejas



Un **estado compuesto** es aquella en la que existen subestados, cada uno de los cuales, a su vez, puede ser un estado compuesto. Un estado compuesto está representado por al menos, dos compartimentos separados por una línea; en la parte superior el nombre, y abajo el que contiene su diagrama de subestados. Cada secuencia de subestados ocupará una franja horizontal separada de las adyacentes por una línea discontinua. Detalles de subestados:

Estados compuestos



- Un subestado inicial se representa con un círculo lleno y uno final con un círculo lleno envuelto en una circunferencia.
- Las transiciones entre subestados de la misma secuencia (misma franja horizontal) se representan igual que entre estados; en cambio entre distintas franjas se usan pseudoestados de sincronización, como se puede ver en el ejemplo entre ev5 y ev6. Además el asterisco nos indica que no hay límite de veces en que se puede producir este salto.
- Las transiciones que entran o salen del estado compuesto, considerado como una unidad, se consideran que si entran, tienen como estados destino todos los estados iniciales de todas las secuencias; mientras que si sale, es como si la transición tuviera como estados de origen todos los estados finales de todas las secuencias. Así al salir en Est3 el estado anterior es Sub3 y Sub4. Al entrar desde Est1 entra en SUB1 y SUB2.
- Hay transiciones que tienen como destino un indicador de historia de los estados (en el ejemplo desde EST1 se puede llegar a H*); indica que en una transición de regreso al estado compuesto se vuelve al mismo subestado del que salió la última vez que partió del estado compuesto. Del indicador de historia puede salir también una transición hacia un subestado que será el destino en caso que ninguna vez se haya estado anteriormente en el estado compuesto (en el ejemplo, se llegaría a SUB1).

2. EL DIAGRAMA DE CASOS DE USO

Los diagramas de casos de uso sirven para mostrar las funciones de un software desde el punto de vista de sus interacciones con el exterior y sin entrar en muchos más detalles. Un **actor** es un conjunto de papeles de una entidad exterior en relación el con el sistema software considerado, podemos decir que un actor es la visión que el software tiene de una entidad exterior. Para que un actor sea considerado como tal, debe cumplir dos requisitos:

- Ser autónomo con respecto al software, es decir, que la actividad que desarrolla a partir de la información suministrada por el software no esté subordinada a la de este.
- Mantener relación directa con el software o con entidades exteriores que desempeñan tareas subordinadas a la actividad del software..

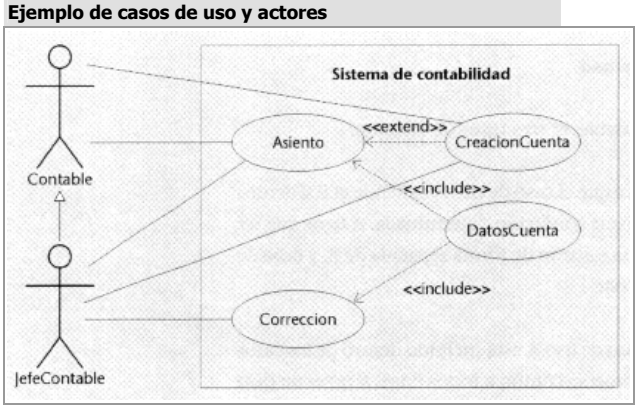
EL **caso de uso** es una documentación sobre una interacción entre el software y un actor o más, siendo esta interacción una función autónoma dentro del software. Las relaciones que se pueden establecer entre casos de uso son:

- Relaciones de **extensión**: El caso de uso A extiende el B si dentro de B se ejecuta A cuando se cumple una condición determinada.
- Relaciones de **inclusión**: Un caso de uso A está incluido dentro de los casos de uso B, C, etc… si es una parte de proceso común a todos estos.
- Relaciones de **generalización/explotación**: Un caso de uso A es una especialización de otro B, si A realiza todo el proceso de B más algún proceso específico.

Tanto para los casos de uso como para los clasificadores, no se utiliza el símbolo de los clasificadores, sino símbolos especiales como los siguientes:

- Los casos de uso se presentan mediante elipses de trazo continuo. A veces todas las elipses se agrupan dentro de un gran rectángulo que representa todo el software.
- Los actores se presentan mediante una figura humana.
- Las relaciones de especialización entre actores y entra casos de uso se presentan igual que en las clases.
- Las relaciones de extensión e inclusión se representan mediante las palabras clave extend e include, respectivamente.

Ejemplo de casos de uso y actores



3. LOS DIAGRAMAS DE INTERACCIÓN

Un diagrama de interacción sirve para describir el funcionamiento de los casos de uso y de operaciones complejas. Para ello, debemos estudiar las interacciones y colaboraciones y los diferentes diagramas que entre ellos se pueden dar.

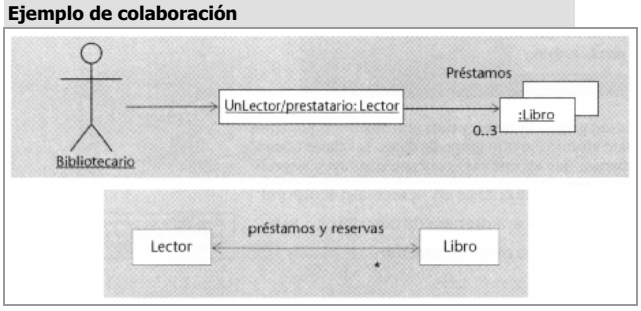
Una **interacción** es la especificación del comportamiento de un caso de uso y operación en términos de secuencias de intercambio de mensajes entre objetos. Estos mensajes contienen estímulos que pueden ser peticiones de ejecución de operaciones o señales.

En cuanto a las **colaboraciones** hay que tener claro primero que es necesario que haya cierta coherencia entre el diagrama estático, para que el diagrama de colaboraciones pueda funcionar. Es decir, las clases y los clasificadores deben formar una correcta infraestructura. Una colaboración es un conjunto de papeles de clasificadores o instancias y de papeles de asociaciones entre aquellos que intervienen en una interacción. La representación gráfica de los papeles es igual que la de los clasificadores y asociaciones correspondientes, aunque solo es necesario especificar los elementos que están modificados en la colaboración.

Los **multiobjetos** representan un conjunto de objetos de un papel con cardinalidad mayor que 1 dentro de una asociación; se representan por dos rectángulos superpuestos y necesitan dos mensajes para realizar una operación en casa uno de sus objetos: uno sirve para seleccionar el conjunto de enlaces de la asociación que corresponden a los objetos; y el otro mensaje se envía separadamente a cada objeto individual por medio del enlace respectivo; a veces estos dos mensajes se combinan dentro de uno.

En la figura lateral observamos un ejemplo de colaboración y debajo el diagrama estático de partida. Se ha definido una colaboración entre un objeto y un multiobjeto y el papel que desempeña el objeto unLector se le ha dado el nombre prestatario, mientras que ni el multiobjeto de la clase Libro ni su papel tienen nombre.

Ejemplo de colaboración

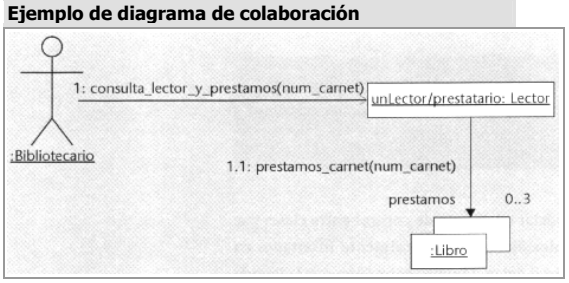


El **diagrama de colaboración** es la representación de una interacción mediante un diagrama estático de la colaboración correspondiente sobre la cual se representan los mensajes de interacción. Para cada mensaje hay una especificación que debe contener:

- **Predecesores**: es la lista de los mensajes predecesores de dicho mensaje. El número de secuencia del mensaje que pone en funcionamiento toda la interacción es 1, a partir de ahí tenemos 1.1., 1.2…. mientras que si el mensaje 1.2. activa un proceso que envía a la vez dos mensajes, uno será 1.2a y otro 1.2b.
- **Guarda**: Es la condición que se tiene que cumplir para que se envíe dicho mensaje.
- **Expresión de secuencia**.
- **Cláusula de condición**: Acostumbra a servir para definir ramas de ejecución.
- **Valores de retorno**: Especifica una lista de nombres de valores retornados (si los hay) como resultado del proceso puesto en marcha por el mensaje.
- **Signatura**: Nombre del estímulo y lista de argumentos entre paréntesis.

A su vez los mensajes pueden ser asíncronos, que es cuando la clase emisora envía un mensaje y se continúa ejecutando sin esperar a que llegue el resultado y se representa por una flecha de punta abierta; o pueden ser síncronos, en los que hay que esperar que el suministrador acepte la respueste y se representa por una flecha de punta coloreada.

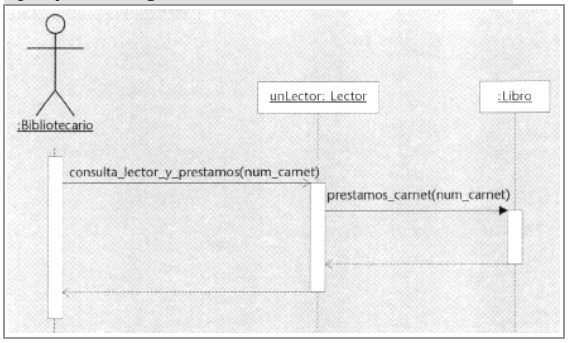
Ejemplo de diagrama de colaboración



En el ejemplo lateral se ha descrito una interacción en la que un bibliotecario pide información sobre un objeto de la clase Lector (identificado por su carnet) y sus préstamos mediante un mensaje al objeto unLector que solo tiene el número de secuencia y la firma de la operación pedida. Durante su ejecución unLector envía un mensaje al multiobjeto de la clase Libro siendo este último mensaje sincrónico.

El **diagrama de secuencias** no representa explícitamente los papeles de asociaciones y se representan explícitamente el orden en el tiempo e incluso la duración de los mensajes y de las operaciones que se ponen en marcha. Se representa en dos dimensiones, con el tiempo se representa verticalmente y corre hacia abajo y en dirección horizontal tenemos los sucesivos papeles de clasificadores que participan en la interacción separados pro franjas verticales discontinuas; así queda representado de cada papel su **línea de vida** en un cierto período de tiempo. Si su destrucción está prevista al final, se indicará marcando con una X. Las **activaciones** son una parte de la línea de vida durante la cual dicho papel ejecuta una acción u otros papeles ejecutan otras acciones, como consecuencia de una acción ejecutada por el papel. Las activaciones se representan mediante rectángulos alargados verticalmente cuyo inicio y final coinciden con la llegada del mensaje que pone en marcha la acción y el envío del mensaje de respuesta, respectivamente. En los mensajes no se indican los números de secuencia, ya que quedan implícitos en el orden temporal de los mensajes; sí se pueden indicar los mensajes de retorno al final de una activación en forma de flechas de línea discontínua y punta abierta.

Ejemplo de diagrama de secuencias



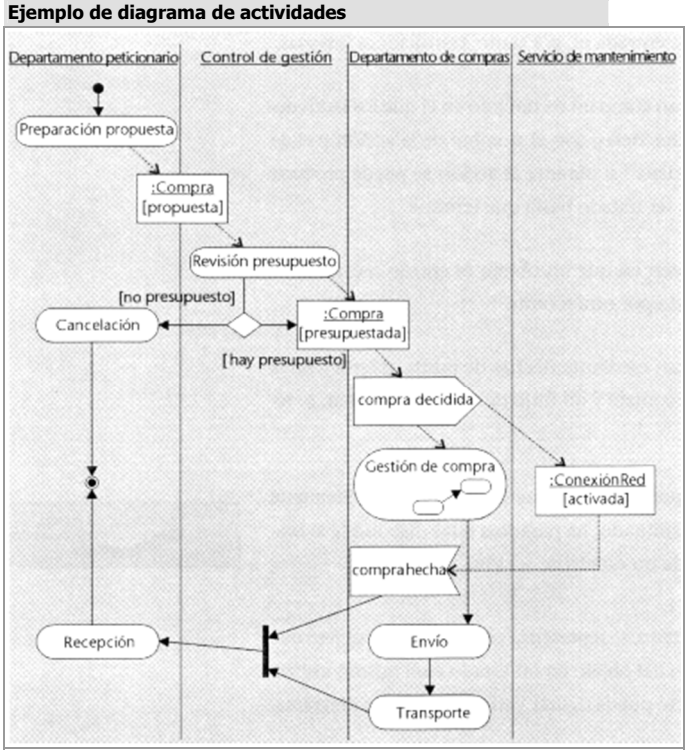
En el ejemplo lateral se observa el diagrama de secuencias correspondiente al ejemplo de colaboración que habíamos visto anteriormente.

4. EL DIAGRAMA DE ACTIVIDADES

Se puede considerar que es una variante tanto del diagrama de estados como el de interacción, ya que sirve para describir los estados de una actividad. Los **elementos específicos** que se encuentran en este diagrama son:

- **Estados de acción**: No hay aquí objetos que esperan a que se produzcan acontecimientos, sino objetos que están desarrollando una acción. Cuando acabe esta acción se producirá la transición, lo cual indica que un estado de acción no puede tener acciones ligadas a otros acontecimientos que no sean el de entrada ni tampoco tienen transiciones internas. Se representan como rectángulos pero con los lados laterales en forma de medio círculo.

Ejemplo de diagrama de actividades



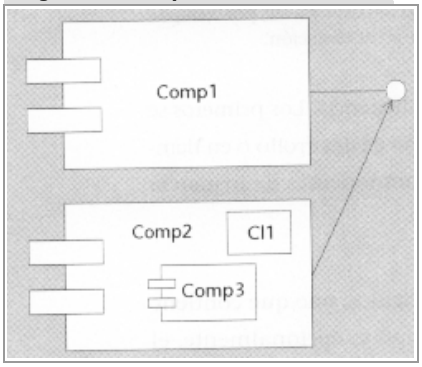
- **Flujos de objetos**: Un objeto es creado o cambia de estado en una acción y es utilizado por otra u otras. Se representan mediante flechas de punta abierta y línea discontinua, como podemos ver en la imagen lateral.
- **Estados de flujo de objeto**: Se representa con un flujo de objeto que entra y otro que sale y el símbolo del objeto en un estado es el mismo utilizado en las colaboraciones (rectángulo); incluso un objeto puede figurar varias veces en un diagrama, cada vez en un estado diferente.
- **Estados de subactividad**: Corresponden a todo un subdiagrama de subactividades y se representan como un estado de acción en cuyo interior hay pequeños estados de acción con transiciones entre ellos.
- **Swimlanes**: Franjas verticales del diagrama que corresponden a unidades organizativas responsables de diferentes acciones (en el ejemplo está el departamento peticionario, el control de gestión, etc.).
- **Iconos de control**: Representan el envío de una señal al acabar un estado de acción y su recepción en otro como entrada. Puede haber bifurcaciones y reunificaciones basadas en condiciones de guarda incompatibles (a menudo complementarias) por lo que no expresan sincronización, ya que los dos flujos de control son alternativos. Se suelen poner rombos en estos iconos de control, como vemos en la unidad organizativo control de gestión, en el ejemplo.

5. LOS DIAGRAMAS DE IMPLEMENTACIÓN

Los diagramas de implementación no describen la funcionalidad del software como hemos visto hasta ahora, sino una estructura general con vistas a su contrucción, ejecución e instalación, y son dos distintos: el diagrama de componentes y el diagrama de despliegue.

El **diagrama de componentes** describe la descomposición física del sistema software en componentes a efectos de su construcción y mantenimiento. Los componentes de los que se compone (valga la redundancia) identifican objetos físicos que hay que tiempo de ejecución, de compilación o de desarrollo, y tienen una interfaz propia y bien definida. Pueden ser códigos fuente ejecutables, DLL, imágenes, incluso documentos manuales. Como vemos en la imagen lateral los componentes se representan mediante tres rectángulos, uno contiene el identificador del componente y dos menores incrustados en el lado izquierdo del primero.

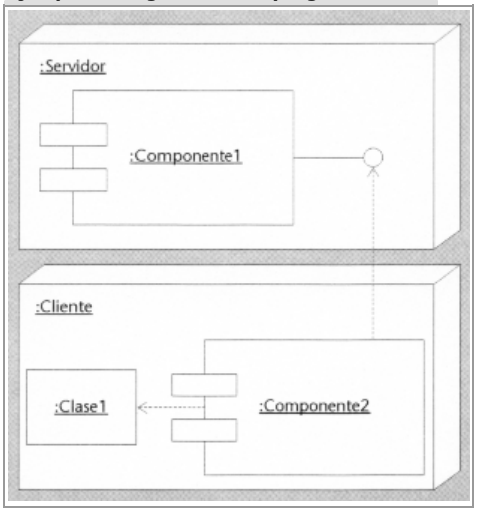
Diagrama de componentes



Las relaciones que se pueden dar entre componentes son **en tiempo de desarrollo** (asociaciones de componentes que modelan dependencias que se tendrán en cuenta en tiempo de compilación o enlace) o **relaciones de llamada** (asociaciones que sirven para modelar llamadas entre componentes. Además también un componente puede tener relaciones de agregación y composición, esto último reflejado en el ejemplo en el que el componente 2 contiene al componente 3.

El **diagrama de despliegue** permite mostrar la arquitectura en tiempo de ejecución del sistema respecto a hardware y software. Se utiliza en el diseño y la implementación y se distinguen en él componentes (como en el diagrama de componentes anterior) y nodos, así como relaciones entre estos. Es más limitado que el diagrama de componentes pues solo representa la estructura del sistema en tiempo de ejecución pero no en tiempo de desarrollo o compilación, pero en su parece es más amplio pues puede contener más clases de elementos.

Ejemplo de diagrama de despliegue



Los **nodos** representan objetos físicos existentes en tiempo de ejecución y modelan recursos que tienen memoria y capacidad de proceso, pueden ser tanto ordenadores como dispositivos o actores. Se representan los nodos mediante paralelepípedos rectangulares, como vemos en el ejemplo lateral. Se establecen entre nodos relaciones mediante líneas continuas que significa que existe comunicación entre ellos. Un componente u objeto se podrá ejecutar si se utilizan los recursos de un nodo o puede estar contenido en éste.

TEMA 4 RECOGIDA Y DOCUMENTACIÓN DE REQUISITOS

Tema 4
Recogida y documentación de requisitos

1.Los requisitos y las fuentes de información
2. Pasos de la recogida y documentación de requisitos
3. La recogida y documentación de requisitos de la interfaz de usuario

1. LOS REQUISITOS Y LAS FUENTES DE INFORMACIÓN

Los **requisitos** son la especificación de lo que debe hacer el software, son descripciones del comportamiento, propiedades y restricciones del software que debemos desarrollar. El papel de los requisitos es el de servir de base para un acuerdo entre los usuarios y los desarrolladores del software (en este sentido debe estar realizado de una manera inteligible para los usuarios que deben revisarlo) y además son la información de partida para desarrollar el software, es decir, son el pie que da entrada a la siguiente etapa.

Hay dos **clases de requisitos**: Los requisitos funcionales describen qué debe realizar el software para sus usuarios, mientras que los no funcionales no van asociados a casos de uso en concreto y son restricciones impuestas por el entorno y la tecnología.

Las **fuentes de información** a las que debemos recurrir para recoger la información sobre los requisitos son:

- **Entrevistas** y eventualmente **encuestas** a los futuros usuarios, si además podemos observarles directamente en su puesto de trabajo, mucho mejor.
- **Documentación sobre el sistema actual** existente, y si está informatizado también echaremos mano de los manuales y procedimientos.
- **Colegas de los usuarios** para conocer el mismo trabajo desde otros puntos de vista.
- **Revisión de sistemas parecidos** dado que el usuario no mencionará funciones importantes del software que para él no son visibles o no le da la debida importancia.

2. PASOS DE LA RECOGIDA Y DOCUMENTACIÓN DE REQUISITOS

Conocemos Los siguientes pasos:

1. El **contexto del software**: Se trata de recoger los requisitos conociendo e indagando en la actividad profesional de los usuarios. Informáticamente sabemos trabajar, pero no conocemos el entorno organizativo que nos piden para ese software. A su vez existen dos modelos para describir este contexto. El **modelo del dominio** es la más simplificada y recoge los tipos de objetos más importantes para establecer una clasificación. Un modelo más complejo es el **modelo de negocio** que describe los procesos y entidades principales entorno al software. En éste último modelo se profundiza identificando todos los casos de uso, las entidades que participan en él…
2. Los **guiones**: Se denomina así a lo que los usuarios conocerán como casos de uso, es decir, identifican de manera organizativa lo que quieren que haga el software; son una fuente de información muy importante, ya que expresan las necesidades de los usuarios tal como ellos las ven.
3. **Identificación de los actores**: Distinguimos principalmente el usuario final (colectivo o personas que interactuarán directamente con el sistema), el usuario privilegiado (o gestor del sistema, encargado de definirla forma de funcionamiento del sistema) y el entorno informático (interfaz no humana del software).
4. **Identificación de los casos de uso**: Se obtienen a partir de los guiones, y son procesos autónomos iniciados por un actor o por otro caso de uso.
5. **Identificación de las relaciones entre casos de uso**: Ya sabemos que hay 3 tipos de relaciones principales: las relaciones de extensión (siempre se relacionan con una condición, como pueden ser diferentes flujos de proceso o casos especiales o errores que debemos tratar); las relaciones de inclusión (es un recurso para evitar la descripción de una misma parte del proceso dentro de

diferentes casos de uso); y la relación de especialización (un caso de uso es versión especializada de la otra).

6. **Identificación de las relaciones de especialización entre actores**: El actor A e suna especialización del B si A tiene todos los apeles de B y alguno más. Esto no suele representar problemas para identificarlo.
7. **Documentación de los casos de uso**: Encontramos dos tipos de documentación: la textual y la formal. En la **documentación textual** encontramos la descripción textual donde aparece el nombre de los actores, otra terminología que se pueda usar con ellos y referencias a otros casos; y también contiene un glosario, muy conveniente para unificar terminología y su interpretación. Por otro lato también está la **documentación formal** que consiste en un diagrama de los casos de uso que muestre todas las relaciones posibles entre éstos y entre los actores; pero no entramos en demasiada profundidad, es decir, solo los utilizamos como función complementaria y no se elaboran sistemáticamente para todos los casos de uso. Más adelante, utilizaremos los diagramas de análisis en esta fase de análisis con mayor profundidad.

3. LA RECOGIDA Y DOCUMENTACIÓN DE REQUISITOS DE LA INTERFAZ USUARIO

La interfaz de usuario es lo que los usuarios ven del funcionamiento del software. También se denomina interfaz hombre-máquina y de ella dependen varios factores:

- La comodidad del usuario en cuanto a ansiedad, frustración, fatiga.
- Productividad del usuario: ergonomía en cuanto a la hora de seleccionar menos teclas y botones.
- Imagen del software: El usuario suele juzgar la calidad del software sólo por la imagen de éste, y nunca se cuenta la calidad de la programación.

No se puede diseñar correctamente una interfaz de usuario sin saber para quién se diseña y hay que evitar el error de diseñar la interfaz para uno mismo, dado que la cultura profesional y los conocimientos informáticos no van a ser los mismos en nuestro caso que en los usuarios reales del software. Para ello es fundamental documentar las **tareas** de los usuarios: es decir, que tareas hacen y con qué frecuencia, el entorno en que se lleva a cabo (limitaciones de espacio, iluminación…), su situación dentro del flujo de tareas, qué documentos y herramientas son necesarios, identificar los problemas y errores más frecuentes.

Por último, las **especificaciones de usabilidad** son los requisitos no funcionales relativos a la interfaz de usuario, y se pueden referir a la facilidad de utilización o aprendizaje o a rapidez y precisión en la ejecución de las tareas, conviene expresarlas en forma cuantitativa, como por ejemplo, que el 90% de los usuarios tras una semana de aprendizaje, puedan registrar una media de diez facturas en 30 minutos o menos.

TEMA 5 ANÁLISIS ORIENTADO A OBJETOS

Tema 5
Introducción a la ingeniería del software orientado a objetos

1. El papel del análisis
2. Paquetes de análisis, y de servicios y revisión casos de uso
3. Especificación de las clases de análisis
4. Especificación formal de los casos de uso y análisis de la interfaz de usuario.

1. EL PAPEL DEL ANÁLISIS

El análisis tiene 3 principales cometidos:

- Traducir los requisitos a un lenguaje más formal. En la etapa anterior de recogida de documentación y requisitos se han descrito las funciones del software en forma de casos de uso y de tareas, ahora bien, un desarrollo posterior orientado a objetos debe utilizar un formalismo de clases y objetos que todavía no tenemos.
- Identificar las clases fundamentales para la implementación del software.
- Expresar los casos de uso en términos de clases.

Existe un alto grado de continuidad entre el análisis y la etapa posterior de diseño, tal es así que probablemente la mayoría de las clases identificadas en la etapa de análisis serán ya definitivas en el diseño posterior. La utilidad del análisis por lo tanto es fundamental pues se puede analizar todo el software con un coste relativamente bajo; hay veces que no se desarrolla un modelo de análisis, es decir, de los requisitos se pasa directamente al diseño, pero solo es recomendable en casos extremadamente sencillos.

2. PAQUETES DE ANÁLISIS Y DE SERVICIOS Y REVISIÓN CASOS DE USO

Por sencillo que sea un proyecto de software, es conveniente dividir la documentación en paquetes de UML. Cada paquete debe ser coherente (los elementos que constituyen el paquete deben estar fuertemente relacionados) y poco dependiente (pocas conexiones entre elementos de paquetes diferentes). A su vez este paquete de análisis se divide en paquetes de servicio, es decir, por su funcionalidad y cada paquete de servicio sólo puede estar en un paquete de análisis a la vez.

Los paquetes de análisis corresponden cada uno de ellos a uno o varios subsistemas enteros y pueden servir para repartir el trabajo del análisis entre varias personas o equipos. Normalmente, los casos de uso entre los que hay relaciones de extensión, inclusión o generalización deben asignarse al mismo paquete.

Los paquetes de servicios son subdivisiones de los paquetes de análisis desde un punto de vista que podríamos llamar *comercial*. Comprenden normalmente casos de uso relacionados y habitualmente tienen que ver con un único actor o, en cualquier caso, con pocos; son indivisibles (o un cliente tiene un paquete de servicios completo, o no lo tiene).

Hay veces que es necesaria una **revisión de los casos de uso**, especialmente si en la fase anterior solo se implementaron estos casos de uso para servir de acuerdo entre usuarios y desarrolladores. Si no se han estudiado a fondo estos casos, puede ocurrir que haya dos casos que realicen funciones parecidas quizá de forma contradictoria.

3. ESPECIFICACIÓN DE LAS CLASES DE ANÁLISIS

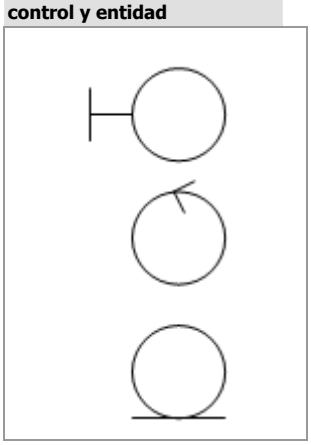
Se consideran **tres tipos de clases de análisis**:

- **Clases de frontera**: Representan en el nivel de análisis la interfaz de usuario por pantalla. Debe haber al menos una para cada papel de cada actor y representan objetos gráficos complejos como ventanas, diálogos de pantalla, menús… Si modificamos en el futuro la forma de presentar en pantalla los

datos pero sin afectar al proceso de los mismos, solo habremos de cambiar las clases frontera.

- **Clases de entidades**: Corresponden a los objetos del dominio que formalmente serán persistentes, es decir, durarán más que el proceso que los crea.
- **Clases de control**: Corresponde a objetos internos del software, no persistentes, no modelan nada del mundo real y sus operaciones suelen contener la parte principal de los algoritmos de aplicación. Si en el futuro tenemos que variar los algoritmos de proceso sin cambiar nada en los datos, solo será necesario modificar las clases de control.

Clases de análisis: frontera, control y entidad



Uno de los problemas que se da es el **identificar las clases de entidades**, es importante hacerlo correctamente porque la mayoría pasarán a la fase de diseño, existen algunas formas de identificarlas, pero no hay una regla maestra, por ejemplo: bibliotecas de clases ya definidas y programadas que además podremos reutilizar; clases sugeridas por los expertos en el dominio, la documentación revisada de los casos de uso…

Normalmente rechazaremos las clases sin atributos, las que no tengan operaciones, las que solo tengan un atributo (ya que pueden ser realmente el atributo de otra) y las clases con un solo objeto.

La **especificación de los atributos** de las clases de entidades es una tarea más sencilla, pues suelen ser datos mencionados explícitamente en los casos de uso. EL modelo de análisis conviene que sea independiente del lenguaje de programación, así que los tipos de atributos serán abstractos (no por ejemplo entero) ni asociados a un lenguaje en concreto; asimismo el nombre de los atributos puede ser libre, pero conviene que tenga un formato compatible con la mayoría de lenguajes de programación. Como casos especiales de atributos encontramos los que tienen un valor compartido por varios objetos (atributos de clase); los atributos no aplicables a todos los objetos de la clase (entonces es mejor crear una superclase con estos atributos aplicables y luego una subclase más restringida); y atributos con valores repetidos (a veces mejor los definimos como una clase aparte con una asociación n a 1 con al primera).

Llegados a este punto, tenemos una lista, en principio completa, aunque provisional de las clases de software y conviene ahora examinar las relaciones de herencia, asociaciones y relaciones de agregación que pudiésemos encontrar:

- **Jerarquías de herencia**: Vamos a encontrar dos, la especialización y la generalización. En la **especialización** solo añadiremos subclases si sus atributos tendrán atributos, operaciones o asociaciones que no tendrá el resto de los objetos de la superclase. En la **generalización** hay más puntos a tener en cuenta: Si diferentes clases de significado parecido tienen atributos y operaciones comunes, podremos definir una superclase común que agrupe a estas subclases, los atributos eso sí deben estar definidos iguales, pero las operaciones solo deben tener el nombre en común (la superclase será abstracta); no tiene sentido que una superclase abstracta tenga sólo una subclase, en este caso suprimiremos la superclase.
- **Herencia múltiple**: Cuando una clase hereda de diferentes superclases, tiene atributos, operaciones, asociaciones y agregaciones de todas éstas. Puede haber conflicto dado que para un mismo atributo o propiedad con el mismo nombre no sabemos que superclase prevalece e incluso existen lenguajes que no soportan la herencia múltiple, aunque esto último no es motivo para renunciar a ella en esta fase de análisis.
- **Interfaces**: Recordemos que son clases abstractas sin atributos y con todas las operaciones abstractas, y pueden ser una buena alternativa a las clases abstractas, ya que son más flexibles.
- **Asociaciones**: Existirá asociación entre clases cuando los objetos de una necesitan la colaboración de objetos de otra para llevar a cabo sus operaciones.

Por ello hay asociaciones entre clases de frontera y al menos algunas clases de control, por una lado, y entre clases de entidades por otro. Conviene una clase asociativa cuando la asociación tiene atributos o bien hay una clase cuya vida de los objetos está relacionada con la de los enlaces de la asociación y solo tiene un objeto para cada enlace. Además conviene revisar casa asociación con cardinalidad múltiple en los dos papeles para determinar si no debería ser en realidad una clase asociativa.

- **Agregaciones**: Son un caso particular de asociación, en el que un papel es parte del otro en algún sentido.

La **identificación de las clases de frontera, control y operaciones** salen de la descripción textual de los casos de uso. Más directamente lo hacen las clases de frontera, más indirecta las clases de control, y las entidades muchas veces están implícitas pero suelen ser bastante obvias.

4. ESPECIFICACIÓN DE CASOS DE USO Y ANÁLISIS INTERFAZ DE USUARIO.

Hasta aquí, hemos descrito la **estructura estática** del software que se desarrolla, a través de la descripción formal de los casos de uso. Pero la forma simplificada que hemos utilizado no tiene en cuenta la secuencia de los mensajes, ni el hecho de que puedan ser repetidos un cierto número de veces o que estén sujetos a diferentes condiciones. Para algunos casos sencillos lo anterior puede ser suficiente, pero en otras ocasiones será necesario realizar un diagrama de secuencia o de colaboración completos, o, a veces, diagramas de actividades.

El **análisis de la interfaz de usuario** parte de la información siguiente:

- Documentación de las tareas futuras.
- Especificaciones de usabilidad.
- Lista de las clases frontera

Y su principal objetivo es un esquema del contenido de cada ventana asociada a una clase de frontera, excepto las ventanas secundarias, que solo sirven para aspectos como presentar mensajes al usuario o pedirle confirmaciones.

TEMA 6 DISEÑO ORIENTADO A OBJETOS

Tema 6
Diseño orientado a objetos

1. El papel del diseño
2. La reutilización
3. El diseño arquitectónico
4. El diseño de los casos de uso
5. Revisión del diagrama estático del diseño
6. Diseño de la persistencia
7. Diseño de la interfaz gráfica del usuario
8. Diseño de los subsistemas

1. EL PAPEL DEL DISEÑO

Recordemos que siguiendo el ciclo de vida del Rational Unified Process, el análisis y el diseño constituyen un único componente de proceso, pero nosotros los consideraremos dos etapas diferentes porque tienen una finalidad distinta y porque el resultado también es claramente diferente.

La etapa del diseño hace de puente entre el análisis yla realización. El modelo del análisis describe las funciones que tiene que desempeñar el software (plantea el problema) mientras que las etapas siguientes deben buscar las soluciones a esas especificaciones.

Hemos visto que en proyectos muy sencillos se puede llegar a prescindir del análisis. En cambio, el diseño es siempre necesario.

2. LA REUTILIZACIÓN

La reutilización es fundamental en el diseño del software y por sí misma, es ya motivo más que suficiente para justificar la existencia de esta fase. Encontramos que con una buena reutilización conseguimos varias ventajas:

- Se abrevia el desarrollo del software, disminuye el trabajo de mantenimiento y mejoramos la fiabilidad.
- El código reutilizado suele ser más eficiente porque se ha ido mejorando con el tiempo.
- Se gana en coherencia: dado que se suelen reutilizar varias clases y por ello suelen estar programadas de la misma forma y con el mismo estilo.
- Si se reutiliza un buen fragmento de código, será mucho más difícil que se pierda ya que habrá muchas más copias.

La reutilización tiene 4 modalidades: la de clases, de componentes, los patrones y los marcos. La **reutilización de clases** suele acontener cuando éstas son independientes del dominio como interfaces gráficas y estructuras de datos generales. La **reutilización de componentes** requiere primero saber qué es un componente: Es un conjunto de clases cuyos objetos colaboran en tiempo de ejecución con el fin de llevar a cabo una función concreta. Dado que un componente implementa una interfaz determinada (todas las operaciones de sus clases que se pueden pedir del exterior) se puede sustituir un componente por otro que implemente la misma interfaz.

Los **patrones** son una manera organizada de recoger la experiencia de los diseñadores de software para volverla a utilizar en casos parecidos. Cuando un experto informático inventa una solución, rara vez la invención es completamente suya, sino que adapta parte de una solución que ya existía. Esto son los patrones, no son en sí programas sino ideas de diseño extraidas de la experiencia que constituyen un esbozo de solución o receta para problemas parecidos que pueden aparecer con frecuencia, y que solo debemos adaptar a nuestro caso particular.

Así la reutilización avanza un paso más; en casos determinados en los que no se puede reutilizar código, al menos se puede reutilizar el diseño, como mínimo, las ideas básicas. Ya existen "recetarios" de patrones preexistentes que, además, van aumentando con el uso de sí mismos, así encontramos que patrones nuevos ya hacen referencia a patrones existentes anteriormente a ellos.

Las **características de los patrones** son:

- Recogen la experiencia, es decir, no se inventan completamente de nuevo.
- Son algo más amplio que una clase u objeto.

- Crean vocabulario: dentro de un diseño se hará referencia a los patrones por su nombre; por ello el nombre que le asignemos al patrón y por el cual se conocerá no debe ser trivial.
- Son un instrumento de documentación, ya que hacer referencia un patrón dentro de nuestra documentación nos ahorrará describirlo enteramente.
- Dan una solución genérica a un caso concreto, que nosotros debemos completar.

Los **componentes de los patrones** son:

- **Nombre**: Ya vimos que no es trivial, en los patrones no puede haber sinónimos o alias.
- **Contexto**: El entorno dentro del cual se presenta el problema que es resuelto por el patrón.
- **Problema**: Requisitos que la solución debe cumplir.
- **Solución**: El patrón.

Los patrones, a menudo, hacen referencia a otros patrones, ya sea porque los utilizan o porque los complementan. Cuando utilizamos un sistema de patrones (bastante frecuente) hay que documentar las relaciones entre éstos y la descripción de las dependencias entre ellos cuando se utilizan juntos.

Los **marcos de aplicaciones** son un conjunto de clases que constituyen una aplicación incompleta y genérica. Si el marco se complementa de manera adecuada (se especializa) se obtienen aplicaciones especializadas de un cierto tipo. Hay dos tipos de marcos:

- Marcos de caja blanca: Con un conjunto de clases de las cuales está definida la interfaz y la implementación. Para especializarlo hay que implementar las subclases de estas clases.
- Marcos de caja negra: Tienen definidas unas interfaces denominadas papeles, y para especializarlo hay que añadirles clases que implementen sus papeles.

Los marcos tienen varias ventajas: Reducen el trabajo al programar, dado que aplicamos subsistemas que sabemos que funcionan y por ello el código no se deberá sobrescribir ni mantener; llevan a desarrollar pequeñas aplicaciones que encajan dentro de los marcos, en lugar de aplicaciones monolíticas; los marcos permiten que otras compañías puedan suministrar componentes que los desarrolladores podrán añadir. Pero a su vez, también encontramos inconvenientes: Limitan la flexibilidad pues los componentes para un marco deben amoldarse a las restricciones impuestas por la arquitectura de éste; tienen un aprendizaje difícil, como media se requieren 3 semanas para estudiarlo a fondo, aunque ciertamente solo se debe realizar una vez y puede servir este aprendizaje para muchas aplicaciones que se basen en este marco estudiado; reducen el grado de creatividad de los desarrolladores.

Aunque los marcos y los patrones pudieran parecernos lo mismo, son bastante diferentes:

- Los patrones son soluciones más pequeñas que un marco: un marco puede contener varios patrones, pero no al revés.
- Los patrones son más abstractos.
- Los patrones son menos especializados, ya que se pueden utilizar en cualquier tipo de aplicación.

3. EL DISEÑO ARQUITECTÓNICO

El diseño arquitectónico tiene como objetivo definir las grandes líneas del modelo del diseño; y comprende las actividades siguientes:

- **Establecimiento de la configuración de la red**: Es decir, determinar los nodos y sus características, las conexiones entre ellos, ancho de banda…

- **Establecimiento de los subsistemas**: Pueden ser subsistemas propios o desarrollados por otras compañías o incluso software de mercado. Podemos también aplicar patrones arquitectónicos y generalmente aplicaremos la división de paquetes realizada en la etapa de análisis. Las dependencias entre subsistemas serán como mínimo, las dependencias que ya existen entre los paquetes de análisis y los paquetes de servicios correspondientes.

4. EL DISEÑO DE LOS CASOS DE USO

El diseño de la implementación de los casos de uso parte del diagrama de colaboración resumido que se ha hecho en la etapa de análisis, y se consideran por separado las clases de frontera, de entidades y de control.

Las clases de frontera son el punto de partida del diseño de la interfaz de usuario. Las clases de control y de entidades sirven para implementar la funcionalidad de los casos de uso. Debemos intentar aprovechar la oportunidad de reutilizar componentes y aplicar patrones y marcos siempre que sea posible.

El proceso de diseño de las clases de control comienza con el estudio de la implementación de las operaciones ya identificadas en el análisis, una por una. A menudo necesitaremos nuevas operaciones de clases para implementar éstas o incluso clases nuevas. Después habrá que estudiar la implementación de estas nuevas operaciones. Con las nuevas clases y operaciones llegamos al **diagrama estático de diseño**.

5. REVISIÓN DEL DIAGRAMA ESTÁTICO DE DISEÑO

Generalmente el diagrama estático de diseño se va haciendo a la par que se diseñan los casos de uso, y una vez acabado ello, debemos hacer una revisión del diagrama obtenido. La revisión del diagrama obtenido (que suele contener 5 veces más clases que el diagrama de análisis) se basa en:

- **Normalización de los nombres**: Por continuidad será bueno utilizar la misma terminología del diagrama estático del análisis. Aunque es posible que debamos cambiar algunos nombres bien porque vulnerar alguna norma aplicable al proyecto, bien para respetar una terminología ya establecida en proyectos anteriores o bien porque el lenguaje de programación no lo soporta (esto último debíamos haberlo previsto con anterioridad.

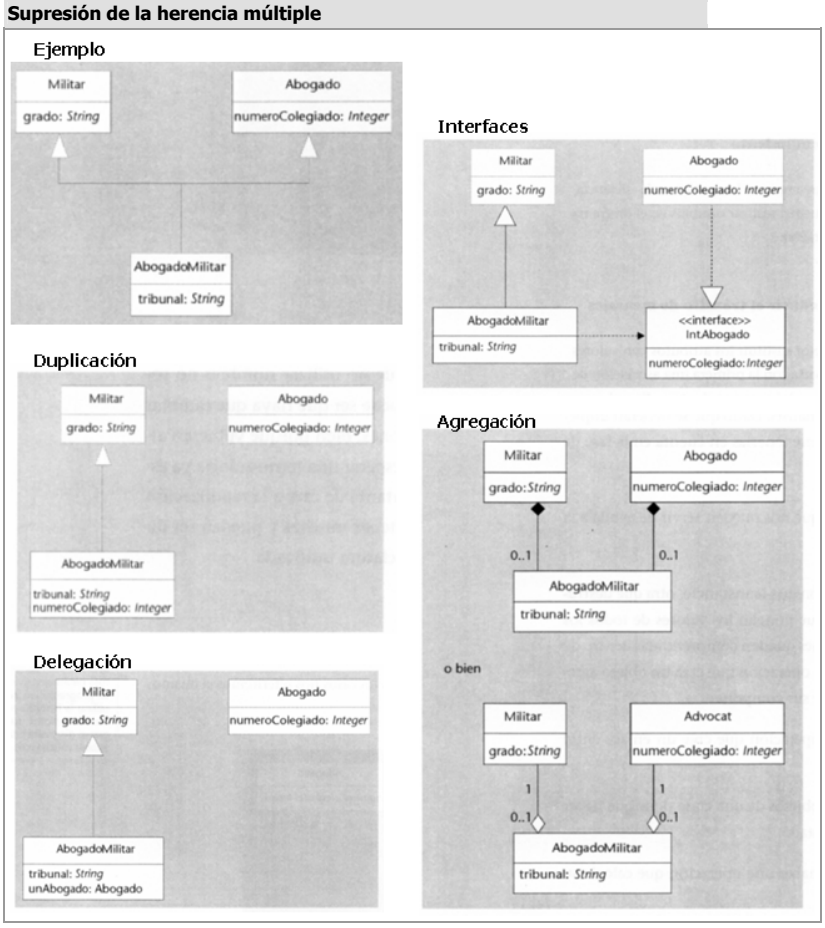
- **Reutilización de las clases**: Durante el diseño las clases se hacen a medida, ahora se trata de revisarlas sistemáticamente en busca de clases que pueden ser reutilizadas por otras ya existentes.

- **Adaptación de la herencia en el nivel soportado por el lenguaje de programación**: La herencia múltiple no suele ser soportada por todos los lenguajes. Por ello existen varias formas de deshacerla: por **duplicación** consistente en duplicar en la subclase los atributos que heredaría de una de las superclases; por **delegación**, manteniendo una sola de las superclases e incluyendo dentro de la subclase una referencia a un objeto de la otra superclase, que tiene el valor de los atributos correspondientes; por **interfaces**, sustituyendo la herencia doble por herencia simple más una interfaz implementada por otra superclase; o por **agregación**, como podemos observar en la imagen de la página siguiente.

- **Sustitución de las interfaces**: Si el lenguaje de programación no soporta las interfaces, las sustituiremos por clases abstractas.

- **Cambios para la mejora del rendimiento**: Ahora en el diseño nos debemos preocupar por cuestiones de eficiencia que en el diseño no nos preocupaban.

Supresión de la herencia múltiple



- **Agrupación de clases para reducir el tránsito de mensajes**: Antes habíamos sustituido los atributos con valores múltiples por una clase aparte que comparta con la primera una asociación de n a 1; pues bien, quizá ahora nos interese el cambio a la inversa, ya que así no hay que enviar un mensaje de una clase a otra.

- **Especificación de las operaciones implícitas**: En general para toda clase debe haber una operación que la instancia, otra que destruya objetos y operaciones que lean y pongan los valores de todos los atributos.

- **Referencias a las clases frontera**: Hay que sustituir las operaciones de las clases de frontera por operaciones en elementos de la interfaz de usuario.

- **Cohesión y acoplamiento**: Ya estamos casi seguro de tener un diagrama estático con todas las clases que deberá tener para sus implementaciones, y con todas sus operaciones bien definidas. Pues es ahora cuando hay que revisarlo para que sea **coherente**; es decir, los atributos y operaciones de cada clase deben tener mucha relación entre sí, hasta el punto de considerarla una unidad. Las clases y operaciones poco coherentes son más difíciles de entender, presuponen relacionar varias entidades en relación de 1 a 1, y se ven afectadas por más modificaciones. El **acoplamiento** también es importante y expresa el grado en el que éstos dependen de otras clases y objetos para llevar a cabo su responsabilidad; es mejor un menor acoplamiento, dado que cuanto más tenga una clase, más se vera afectada por cambios en otras y por tanto, se deberá modificar más a menudo.

6. DISEÑO DE LA PERSISTENCIA

Se llama clases persistentes a las que pueden tener objetos persistentes, y clases temporales a las no persistentes. Un objeto persistente es aquel que tiene una vida más larga que el proceso que lo crea, o dicho de otra forma, un objeto se crea en un proceso y se utiliza en procesos posteriores, por lo que hay que grabarlo en un sistema de almacenamiento permanente y también lógicamente debe poder ser leído. Existen tres tipos principales de sistemas de almacenamiento: bases de datos orientadas a objetos, bases de datos relacionales y ficheros clásicos; y base de datos object-relacional.

Bases de datos orientadas a objetos

Como es lógico suponer es el caso más sencillo de todos, ya que no hay que transformar los objetos para hacerlos persistentes. Solo hay que enriquecer la definición de la clase con objetos persistentes, indicando cuales de sus atributos lo son y qué política de lectura de objetos se requiere: todos de una vez, o leer según demanda).

Bases de datos relacionales y ficheros clásicos

Aquí la cuestión ya se complica un poco: a un objeto le corresponde en principio, una fila de una tabla de base de datos relacional o un registro de un fichero; y a los atributos le corresponden columnas de la tabla correspondiente. Se hace necesario realizar la transformación anterior antes de grabar un objeto y hay que llevar a cabo la inversa antes de poderlo utilizar para ser leído. Existen tres formas de llevar a cabo la gestión de la persistencia:

- Cada clase persistente tiene operaciones para que los objetos se graben, borren, etc, por sí mismos. Es la opción más eficiente dado que requiere menos llamadas a objetos, pero dependerá de cada gestión de base de datos concreta y, por tanto, no es una solución portable a otro sistema distinto.
- Gestor de disco para cada clase persistente: Este gestor accede directamente en cada clase al fichero o base de datos, así la clase de entidades se desacopla del sistema de gestión de base de datos y además el gestor puede servir de memoria caché intermedia. Esta solución es menos eficiente que la anterior.
- Mezcla de las dos anteriores: se crean gestores de disco y se añaden operación de grabación, lectura en cada clase de dominio. Se diferencia del primer método en que puede ser más portable, sobre todo si implementamos la persistencia por medio de un marco.

En cualquiera de los casos debemos pasar por 4 fases para **definir la estructura** de base de datos relacional o con ficheros clásicos:

1. Transformamos el modelo estático en un modelo entidad-relación: se convierte cada clase con un atributo con valores múltiples en dos clases unidas por una asociación, y después le hacemos corresponder un tipo de entidad a cada clase no asociativa. Después se añade una relación para cada asociación con cardinalidades que son obvias.
2. Supresión de la herencia: Como el sistema de base de datos no soporta la herencia podemos resolverlo:
    a. Definiendo una tabla para cada clase que contendrá los atributos propios de la clase y los heredados. Es una solución sencilla pero tiene tantas tablas como subclases, hay que crear un gestor de disco para cada subclase, un cambio en la definición de "cliente" obligaría a cambiar todos los gestores de disco…
    b. Se puede crear una tabla para la superclase en la que se graba a todos los clientes; a la vez se crea una tabla complementaria para cada subclase con el identificador del objeto y los atributos específicos (en este caso el descuento). Es buena solución si sólo una pequeña fracción de los objetos de la clase pertenecen a la subclase y además si los valores de los atributos son de longitud fija.
    c. Se crea una única tabla para todas las jerarquías de herencia con todos los campos, tanto de la superclase como de todas sus subclases y se le añade un campo que nos diga qué subclase del objeto corresponde a cada fila. La eficiencia en tiempo es muy elevada en esta solución, pero menos en ocupación de disco.

Supresión de la herencia

CASO A
Tabla Cliente
Nif: string, clave principal
Nombre: String
….
Tabla ClienteEspecial
NIF: String, clave principal
Nombre: String
Descuento: Integer

CASO B
Tabla Cliente
Nif: string, clave principal
Nombre: String
….
Tabla Auxiliar ClienteEspecial
NIF: String, clave principal
Descuento: Integer

CASO C
Tabla Cliente
Nif: string, clave principal
Tipo: Integer
Nombre: String
….
Descuento: Integer
…

El **gestor de disco** es una clase diferente de la clase persistente y, dentro de las operaciones de ésta, hay llamadas a operaciones del gestor de disco. Podemos considerar que todos los gestores de disco tienen al menos unas operaciones básicas: leer, grabar, regrabar y borrar. Por ello todos los gestores son subclases de una superclase GestorGenerico, en la cual las operaciones serían abstractas.

El diseñar **gestores de disco para la materialización según demanda** (lectura según demanda) es más compleja y sólo se justifica cuando la materialización es costosa o cuando se debe poder sustituir la clase de entidad sin tener que modificar todas las que la utilizan. En resumen las referencias exteriores de estos gestores no se hacen a objetos de la clase de entidades en cuestión, sino

a objetos de una clase sustituta. La clase original y la sustituta implementan la misma interfaz.

Bases de datos object-relational

Es el mismo diseño que las bases de datos relacionales, con la diferencia de que este nievo modelo puede tener atributos de tipo compuesto y, por tanto, no será necesario crear gestores de disco para todas las clases persistentes, sino sólo para aquellas a las cuales se accederá directamente.

7. DISEÑO DE LA INTERFAZ GRÁFICA DE USUARIO

La interfaz gráfica de usuario (GUI) es solo una parte de la interfaz de usuario en general: es la interfaz implementada por medio de pantallas gráficas con teclado y ratón (no entran aquí por ejemplo los listados por impresora) y su diseño considera tres aspectos:

- El **contenido**: Establecido en los esquemas de las ventanas elaborados en el análisis.
- El **formato**: Que se especifica íntegramente en esta fase.
- La **interacción**: Es decir, la dinámica de la interfaz de usuario, denominada diálogo entre usuario y sistema.

En general, y para no extendernos demasiado, las **características** de un buen diseño son:

- Sensación de control: Los usuarios no quieren tener la sensación de ser controlados por el ordenador.
- Adecuación al usuario y no a nosotros que lo diseñamos.
- Familiaridad para el usuario; utilizando conceptos y organizaciones a los que suela estar acostumbrado.
- Flexibilidad: Interfaz adaptable a las preferencias del usuario.
- Atención a las posibilidades de errores del usuario.
- Robustez y sensación de seguridad: Se tiene que permitir que el usuario explore el producto sin riesgo de perder información o de perderse, permitiendo siempre deshacer la última acción.
- Facilidad de utilización y aprendizaje

8. DISEÑO DE LOS SUBSISTEMAS

Habíamos definido con anterioridad subsistemas en el diseño arquitectónico y se habían especificado las interfaces respectivas y las dependencias entre éstas. Cuando ahora se acabe la fase de diseño, conviene llenar de contenido los subsistemas especificando qué clases comprende cada uno y haciendo explícitas las dependencias en el nivel de clase.

TEMA 7 INTRODUCCIÓN AL SOFTWARE DISTRIBUIDO

Tema 7
Introducción al software distribuido

1. Entornos distribuidos y entornos abiertos
2. Entornos cliente/servidores clásicos
3. Entornos con middleware: CORBA
4. RMI
5. Documentos compuestos distribuidos: DCOM
6. Desarrollo del software distribuido

1. ENTORNOS DISTRIBUIDOS Y ENTORNOS ABIERTOS

Se habla de **entorno distribuido** cuando el software de una aplicación se ejecuta repartido entre varios ordenadores de una red; entonces también decimos que el software en cuestión es distribuido. La razón decisiva para la evolución hacia los entornos distribuidos es que la misma capacidad de proceso resulta mucho más barata si se consigue con PC que con mainframes o máquinas Unix (incluso 5 veces más barato).

Los objetivos que se persiguen con los entornos distribuidos son:

- Portabilidad de aplicaciones y de servicios del sistema
- Interoperabilidad: En diferentes ordenadores y aplicaciones podemos comunicarnos.
- Integración: Uniformidad.
- Transparencia: Los usuarios pueden leer datos de un ordenador sin saber dónde está y los procesos se pueden ejecutar en varios ordenadores sin que los usuarios sepan en cual de ellos.
- Facilidad de crecimiento del sistema.
- Compartimiento: Se pueden compartir recursos, servicios y aplicaciones.
- Seguridad: Los datos de un usuario están protegidos en lo que respecta a la consulta y actualización.

Un **entorno abierto** es un sistema distribuido en el que se intentan conseguir los objetivos de los sistemas distribuidos, y hacer que las interfaces entres sus componentes respeten un conjunto amplio y completo de normas sobre comunicaciones, programación, presentación e interfaces.

2. ENTORNOS CLIENTE/SERVIDOR CLÁSICOS

La idea básica de la **arquitectura cliente/servidor** es que un programa (el servidor) gestiona un recurso compartido concreto y hace determinadas funciones sólo cuando las pide otro (el cliente) que es quien interactúa con el usuario. Normalmente los programas cliente y servidor se encuentran en ordenadores diferentes.

Las ventajas que proporciona este modelo es que permite que la información se procese cerca de donde se ha generado, las funciones del software pueden quedar repartidas en varias máquinas, el crecimiento del hardware puede ser gradual y se facilita el uso de interfaces gráficas de usuario y aplicaciones multimedia. Los inconvenientes son que el servidor puede ser un cuello de botella, el software distribuido es más complejo y por tanto proporciona más errores y tiende a fallar más.

Encontramos la **arquitectura cliente/servidor de dos capas**:

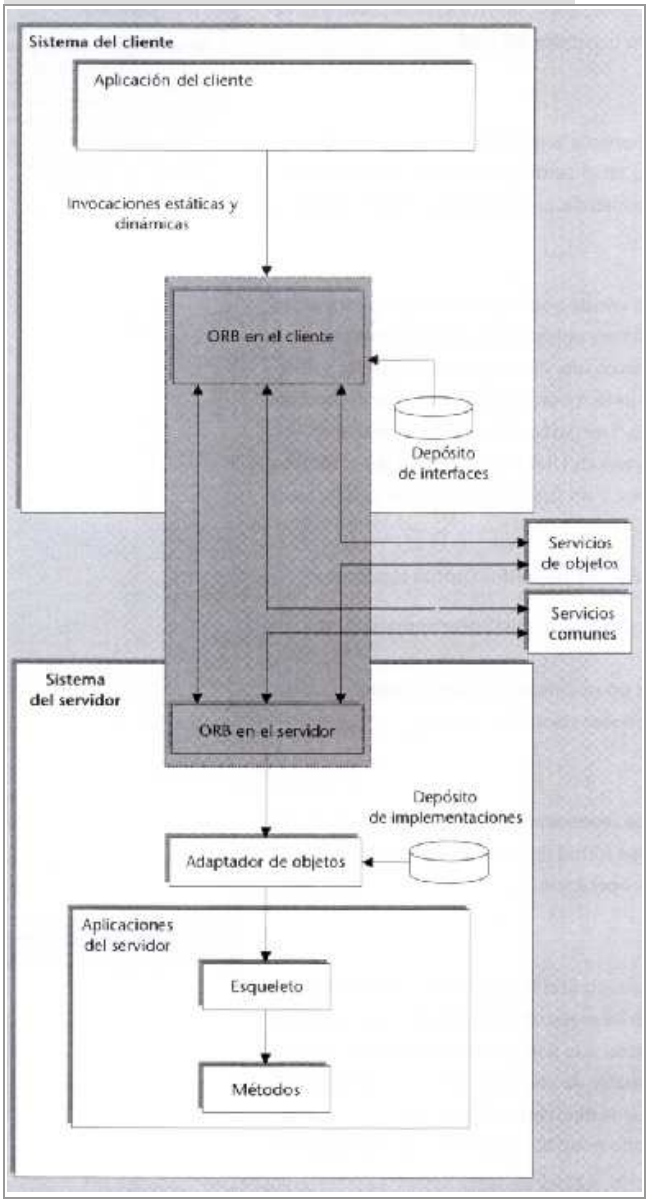
- **Primera generación**: Típico de redes de área local donde los clientes son PC o estaciones de trabajo en los que se ejecutan las aplicaciones. Los servidores solo llevan a cabo funciones generales y de bajo nivel.
- **Entornos de igual a igual**: Un ordenador es el cliente para otras máquinas y el servidor para estas mismas u otras e incluso sí mismo.
- **Segunda generación**.

Las arquitecturas de dos capas presentan limitaciones al crecimiento del número de clientes ya que todos ellos deben conectarse al mismo servidor y es difícil mantener el software de los clientes, ya que cualquier cambio se debe hacer en todos al mismo tiempo.

En la **Arquitectura de tres capas** encontramos un servidor (primera capa) y un agente y un cliente (2ª y 3ª respectivamente). Los agentes intermediarios se encargan de traducir las aplicaciones, controlar la carga del servidor…

3. ENTORNOS MIDDLEVARE: CORBA

Supresión de la herencia múltiple



Cuando la red aumenta todavía más sus dimensiones e incorpora multiplicidad de aplicaciones, plataformas y redes, es necesario un componente que gestione la comunicación entre todos estos elementos; este componente va colocado entre los clientes y los servidores y por este motivo se denomina software intermedio o **middleware**.

El software distribuido puede llevarse a cabo con software clásico, pero si se hace con software orientado a objetos se dan mejores resultados dado que el paso de mensajes a un objeto que está en otro ordenador puede hacerse igual que si estuviera en el mismo.

En CORBA hablaremos de interfaces, sabemos que en el software orientado a objetos y como consecuencia de la encapsulación, los objetos se comunican por medio de mensajes en los que se piden operaciones y valores de atributos y por ello distinguimos interfaces. El concepto de interfaz en CORBA es muy parecido al de UML ya que la misma organización ha especificado ambos.

El procesamiento de las peticiones según la imagen lateral, tiene las siguientes fases:

- El cliente invoca la petición.
- Cuando llega al ORB (Object Request Broker), la demanda incluye una referencia al objeto que es su destinatario. El ORB utilizando el depósito de implementaciones, localiza el proceso del servidor o lo pone en funcionamiento si es necesario.
- El ORB localiza el POA (Portable Object Adapter) correspondiente al objeto destinatario de la demanda o bien pide su creación al activador del adaptador. Si no se consigue se recibirá la excepción object_not_exist.
- El POA localiza o activa el sirviente del objeto de formas distintas según las políticas vigentes.
- Se localiza el esqueleto y se hace la gestión de las respuestas y las excepciones.

Los servicios de objetos que venimos a encontrar en un entorno CORBA son:

- Servicio de nombres: Gestiona estructuras de nombres de objetos que sirven para localizarlos desde otros objetos.
- Servicio de acontecimientos: Un acontecimiento es un hecho que guarda relación con un objeto y tiene interés para otros objetos, a los que se hace accesible mediante un mensaje denominado notificación. Este servicio gestiona este tipo de eventos.
- Servicio de notificaciones: Es una extensión del servicio anterior con funciones adicionales.
- Servicio de relaciones: Permite crear relaciones, permanentes y temporales, entre dos o más objetos, sin modificarlos y sin que éstos lo sepan. El concepto de relación en este servicio es muy similar al de asociación en UML.

- Servicio de ciclo de vida: Proporciona operaciones para crear, copiar, mover y borrar objetos de forma local o remota; además soporta asociaciones entre grupos de objetos y condiciones de integridad referencial entre objetos.
- Servicio de propiedades: Sirve para definir atributos de los objetos dinámicamente.
- Servicio de tiempo: Se utiliza para obtener la hora, confirmar el orden en que se han producido diferentes acontecimientos y generar eventos relativos al tiempo.
- Servicio de externalización: Sirve para convertir un objeto en un stream (externalizarlo) o justo lo congtrario (internalizarlo).
- Servicio de estados persistentes: Su función es guardar el estado de un objeto en memoria permanente y recuperarlo cuando sea necesario.
- Servicio de control de la concurrencia: Su finalidad es arbitrar los accesos concurrentes a un objeto con la intención de garantizar su integridad. Proporciona interfaces para que los clientes puedan reservar y liberar recursos para coordinarse en su uso compartido.
- Servicio de transacciones: Una transacción es una unidad de proceso que o bien llega a acabar normalmente o bien, en el caso de que se vean abortadas, las actualizaciones en bases de datos que hubiesen hecho se anulan, como si nunca hubiesen empezado.
- Servicio de seguridad: Mantiene la confidencialidad, integridad y disponibilidad de los datos mediante identificaciones por autenticación, control de acceso o auditoría de seguridad.
- Servicio de licencias: En el mundo de los objetos distribuidos podemos encontrar objetos de pago; entonces conviene que sea posible medir el uso de los objetos con vistas a una posterior facturación. El servicio de licencias da apoyo a esta función.

4. RMI

RMI (Temote Method Invocation) es la herramienta de Java para el soporte de objetos distribuidos. RMI tiene una funcionalidad muy reducida en comparación con CORBA, pero en cambio presenta la ventaja de que las invocaciones remotas tienen lugar sin salir del entorno Java.

5. DOCUMENTOS COMPUESTOS DISTRIBUIDOS: DCOM

Los documentos compuestos nos interesan porque dentro de una interfaz gráfica de usuario pueden figurar documentos considerados objetos, y porque los documentos compuestos pueden resultar de la combinación de documentos distribuidos y es entonces un caso particular de la tecnología de objetos distribuidos.

Un **documento compuesto** es aquel documento que resulta de la agregación de otros (que además también pueden ser compuestos), que denominaremos **componentes**, dentro de un determinado documento marco.

OLE es una ampliación del portapapeles y de DDE para crear documentos compuestos orientados a objetos. Se puede llevar a cabo la inclusión de documentos de dos formas:

- **Embedding**: Inclusión: Una copia del objeto y su formato de presentación se almacenan dentro del documento y se transfieren con este.
- **Linking**: Enlace: El documento que se incluye dentro del compuesto continúa dentro de su ubicación original en un documento exterior o dentro del mismo documento compuesto, y el objeto que lo representa en este último sólo contiene el formato y un puntero.

Un **documento compuesto de OLE** contiene tantos datos propios como objetos gestionados por otras aplicaciones: típicamente le corresponde un fichero compuesto, y hace de intermediario entre dos tipos de componentes: los contenedores, que proporcionan lugares donde poner las cosas, y los servidores, que son las cosas que se ponen. Un tercer tipo de componentes de OLE son los OCX, que tienen un conjunto de interfaces predefinidas.

6. DESARROLLO DEL SOFTWARE DISTRIBUIDO

El **análisis de requisitos** del software distribuido es el mismo que veníamos haciendo hasta ahora, dado que este análisis se ocupa de lo que le debe aparecer al usuario por las pantallas y por las impresoras, independientemente de que todo el proceso de haga localmente o no.

La **distribución de objetos** también es similar a como la hacíamos hasta ahora, dado que están determinados por la funcionalidad.

Texto elaborado a partir de:
Ingeniería del software
Benet Campderrich Falgueras, Recerca Informàtica, S.L.
Febrero 2004